🧩 Python match Statement (introduced in Python 3.10)

It's similar to **switch–case** statements in other languages like C, Java, or C#.
It makes your **conditional logic cleaner and more readable** when you have many possible values to compare.

🧠 What is match?

match is used to **compare a variable's value** against **multiple patterns**.
Each pattern is checked using case blocks.
When a match is found, the corresponding block of code executes.

🔹 Basic Syntax

```
match variable:
    case value1:
        # code block
    case value2:
        # code block
    case _:
        # default case (like else)
```

🔹 Example 1: Basic Match

```
day = "Monday"

match day:
    case "Monday":
        print("Start of the week")
    case "Friday":
        print("Weekend is near!")
```

```
    case "Sunday":
        print("Time to relax")
    case _:
        print("Just another day")
```

Output:

```
Start of the week
```

💡 _ acts like a **default case** — it matches anything not matched before.

🔹 **Example 2: Multiple Matches in One Case**

You can match multiple values together using | (OR).

```
day = "Saturday"

match day:
    case "Saturday" | "Sunday":
        print("Weekend!")
    case _:
        print("Weekday")
```

Output:

```
Weekend!
```

🔹 **Example 3: Match with Variables (Pattern Matching)**

You can even match **patterns** and **extract values** — like destructuring tuples or objects.

```
point = (0, 5)

match point:
    case (0, 0):
```

```
        print("Origin")
    case (0, y):
        print(f"Y-axis at y={y}")
    case (x, 0):
        print(f"X-axis at x={x}")
    case (x, y):
        print(f"Point is at ({x}, {y})")
```

Output:

```
Y-axis at y=5
```

- **Example 4: Match with Conditions (Guards)**

You can add an **if condition** (called a *guard*) inside a case.

```
num = 10

match num:
    case n if n < 0:
        print("Negative number")
    case n if n == 0:
        print("Zero")
    case n if n > 0:
        print("Positive number")
```

Output:

```
Positive number
```

- **Example 5: Match with Dictionary**

```
person = {"name": "Alice", "age": 25}
```

```
match person:
    case {"name": name, "age": age} if age >= 18:
        print(f"{name} is an adult.")
    case {"name": name, "age": age}:
        print(f"{name} is a minor.")
    case _:
        print("Invalid data")
```

Output:

Alice is an adult.

- **Example 6: Simple Calculator using Match**

```
a = 10
b = 5
operation = "divide"

match operation:
    case "add":
        print(a + b)
    case "subtract":
        print(a - b)
    case "multiply":
        print(a * b)
    case "divide":
        print(a / b)
    case _:
```

```
    print("Invalid operation")
```

Output:

2.0

🧠 Advantages of match Over if-elif

Feature	if-elif	match-case
Syntax	Repetitive	Clean, readable
Pattern matching	❌ No	✅ Yes (tuples, dicts, etc.)
Default case	else	_
Introduced	Since Python 1.x	Python 3.10+